

FSM Interface Specification

The FSM is responsible for machine state management only. It does not parse G-code, control motors, or read hardware. Other modules notify the FSM using processEvent().

Public API

Function	Purpose
begin()	Start FSM in INITIALIZING state.
update()	Called once each Arduino loop.
processEvent(Event e)	Notify FSM of an event.
getCurrentState()	Return current state.
isIdle()	True if state == IDLE.
isBusy()	True if INITIALIZING, HOMING or MOVING.
isFault()	True if state == FAULT.

Events

Event	Generated By
INIT_SUCCESS	Initialization module
INIT_FAILED	Initialization module
G28_RECEIVED	G-code parser
G1_RECEIVED	G-code parser
HOME_COMPLETE	Motion controller
MOVE_COMPLETE	Motion controller
LIMIT_TRIGGERED	Limit switch module
TIMEOUT	Motion controller
RESET	User/UI

States

INITIALIZING → IDLE/HOMING/MOVING/FAULT. The FSM transitions according to received events. Modules should never modify the current state directly.

Typical Integration

loop(): parser.update(); motion.update(); limitSwitch.update(); fsm.update(); Parser sends G-code events, motion controller sends completion/timeout events, limit switch module sends LIMIT_TRIGGERED, initialization sends INIT_SUCCESS or INIT_FAILED.